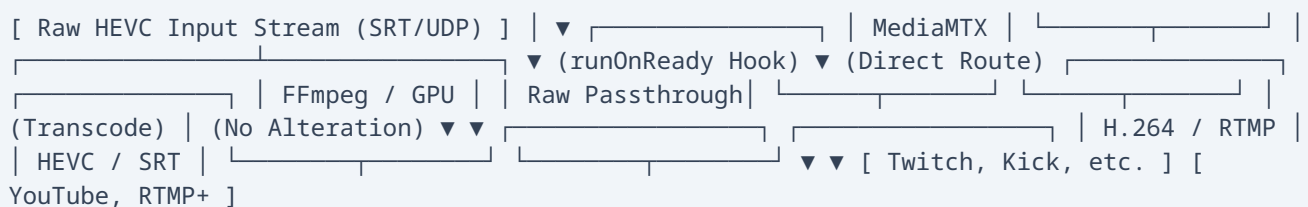


Slimcast Architecture Blueprint

Comprehensive Specifications for Production Ingest & Hardware-Accelerated Transcoding

1. Architectural Pipeline Overview

The system is engineered as a low-latency, high-efficiency cloud transceiver designed to receive a high-quality contribution feed via SRT over UDP, clone it, transcode a copy to H.264 via hardware acceleration for legacy platforms, and pass the raw HEVC stream directly through to supported destinations without altering it.



2. Media Server Core: MediaMTX Integration

Rather than using heavy CDN cluster servers, **MediaMTX** serves as the central protocol translator and process coordinator. It functions as a lightweight, zero-dependency utility that natively bridges SRT and RTMP.

- **Ingest Protocol:** MediaMTX serves as an SRT listener. It opens a dedicated UDP port to await the incoming contribution feed.
- **Automation Hooks:** Using the `runOnReady` configuration directive in `mediamtx.yml`, the server instantly detects an incoming stream on a specific path (e.g., `/live/source`) and spawns a local hardware-accelerated FFmpeg child process.
- **Egress Distribution:** MediaMTX receives the returned H.264 stream locally from FFmpeg and utilizes internal forwarding mechanisms to push to multiple legacy RTMP ingest endpoints simultaneously. Concurrently, it maintains the raw incoming HEVC stream on its original path for platforms capable of direct HEVC ingest.

3. GPU Hardware & Silicon Specifications

Live transcoding high-motion 1440p (2K) video at 60 FPS requires dedicated physical silicon blocks to handle data density without incurring massive latency or CPU exhaustion.

GPU Model	Architecture	NVENC / NVDEC	Technical Advantage
Nvidia L4	Ada Lovelace	2x / 4x	Top Choice. Dual encoders allow massive stream density. Hardware support for AV1, extreme power efficiency. Easily scales to multiple concurrent pipelines.
Nvidia A10 / A10G	Ampere	1x / 2x	Excellent mid-tier workhorse. Massive raw processing power ensuring flawless frame time stability during high-motion visual spikes.
Nvidia T4	Turing	1x / 1x	Highly cost-effective but strictly limited to low-density pipelines. Struggles if handling multiple heavy background tasks concurrently.

The Consumer GPU Lockout (RTX 4060 / 4090): While consumer GeForce cards feature identical physical NVENC/NVDEC silicon blocks to their enterprise counterparts, they are restricted by an artificial driver-level limit of **8 concurrent encoding streams**. Furthermore, Nvidia's EULA legally prohibits the deployment of consumer GeForce cards within commercial data center environments, making them unavailable on Tier-1 cloud providers (AWS, GCP, Azure).

4. The Driver & Hardware Acceleration Stack

To prevent high-motion frames from bouncing between system RAM and GPU RAM (which causes immediate bottlenecking and latency), the entire transcoding ecosystem must execute explicitly inside GPU memory.

```
[ SRT Ingest Packet ] —> [ NVDEC (VRAM Decode) ] —> [ CUDA Surface Memory ] —> [ NVENC (VRAM Encode) ] —> [ RTMP Egress Packet ]
```

Required Linux Driver Layers:

- **Nvidia Proprietary Display Driver:** The foundational kernel module required for the OS to communicate with the physical PCI-e device.
- **CUDA Toolkit:** Mandatory library infrastructure used by transcoding software to allocate structures directly in GPU memory frames.
- **NV-Codec-Headers:** A critical compilation-level package from Nvidia that mapping software requires to expose the underlying Video Codec SDK APIs directly to FFmpeg.

5. Network Stack & Infrastructure Environments

Because SRT relies on UDP to manage selective packet retransmission, standard TCP-optimized cloud infrastructure will choke during high-motion spikes without surgical configuration.

- **RunPod (Incompatible):** RunPod utilizes custom HTTP/TCP load-balancing proxies in front of their containers. They **strictly block all incoming UDP packets**, completely preventing an SRT stream from resolving.
- **AWS / GCP / Azure (Optimal VM Ingest):** These providers offer isolated Virtual Machines with full root access. This allows you to permanently tune the Linux kernel's maximum socket receive buffers (`rmem_max`) to absorb sudden, heavy video data bursts by running: `sudo sysctl -w net.core.rmem_max=26214400`.
- **Vast.ai (Viable Container Marketplace):** Vast.ai allows raw UDP port forwarding but runs in a shared, multi-tenant environment. You cannot modify the host's underlying kernel buffer, and ports are mapped via NAT dynamically.

Vast.ai Precise Networking Configuration:

- **The UDP Flag:** When configuring your instance, you must explicitly declare the protocol inside the Docker options. Omitting the suffix defaults the port to TCP: `-p 9000:9000/udp`.
- **NAT Port Mapping:** Vast.ai will listen on a randomized external public port (e.g., 34821) and route that traffic down to your internal port 9000. You must identify this mapping using the instance control dashboard or by checking the `$VAST_UDP_PORT_9000` environment variable.
- **The Identity Port Configuration:** If your local broadcasting software requires the external port to precisely mirror your internal configuration, request a port designation **above 70000** (e.g., `-p 72000:72000/udp`). Vast.ai reserves this block to force a 1:1 match across their public NAT layer.

6. Production-Ready FFmpeg Blueprint

When operating in containerized environments where you do not possess root authority to optimize the Linux kernel network buffers, you must offload the buffering load entirely to the application layer.

```
ffmpeg -hide_banner -y -fflags +genpts -thread_queue_size 10240 -hwaccel cuda -hwaccel_output_format cuda -i "srt://0.0.0.0:9000?mode=listener&latency=2000000&recv_buffer_size=67108864&rcvbuf=67108864" -c:v h264_nvenc -preset p4 -tune hq -b:v 6000k -maxrate 6000k -bufsize 12000k -g 120 -profile:v high -c:a aac -b:a 160k -f flv rtmp://127.0.0.1:1935/out/live
```

Granular Engineering Flag Breakdown:

- **-thread_queue_size 10240**: Instructs FFmpeg to queue up to 10,240 frames inside software memory between the raw network ingest thread and the hardware decoding engine, preventing a processing lag spike from dropping incoming packets.
- **latency=2000000**: Sets the SRT internal handshake latency parameter to exactly 2,000,000 microseconds (2 seconds). This allows the protocol ample window duration to request and re-transmit dropped UDP fragments over a jittery residential upload link before giving up and presenting visual distortion.
- **recv_buffer_size=67108864 & rcvbuf=67108864**: Allocates a definitive 64 megabyte application-level memory pool strictly to buffer incoming SRT transport stream fragments. This overrides and insulates the pipeline from low host OS kernel allocations.
- **-hwaccel cuda -hwaccel_output_format cuda**: Locks the incoming decoding process and subsequent frames entirely into NVDEC/VRAM surface space. Zero data is written back to host system RAM, minimizing PCIe bus saturation.
- **-preset p4 -tune hq**: Instructs the Gen NVENC encoder to run its multi-pass algorithm with High Quality optimization. This is the exact sweet spot for high-motion gaming content, capturing sudden visual complexity (like grass, explosions, or fast camera panning) without yielding heavy blocky artifacts.
- **-b:v 6000k -maxrate 6000k -bufsize 12000k**: Constrains the output bitrate to a strict 6.0 Mbps ceiling compatible with Twitch/Kick ingestion standards, utilizing a 12 Megabyte VBV encoder buffer to smooth out sudden bitrate variances.
- **-g 120**: Forces a fixed Keyframe/IDR frame interval every 120 frames (exactly every 2 seconds at 60 FPS), satisfying the rigid constraints mandatory for live video delivery networks.